

CSE 144 Final Project Report

Transfer Learning Challenge

Samuel Morrow (CruzID: 193368)

Spring 2026

1 Introduction

This project explores the difficulty of image classification with very limited data. There are only ten labeled images per class in this 100-class dataset, with 1,000 images total. We want to hopefully achieve a high test accuracy on these 1,000 images, using transfer learning.

Transfer learning here is appropriate here because a deep neural network from scratch trained on just 1,000 images would SEVERELY overfit. But, with weight initialization on a model that has already been pretrained on a much larger dataset, that model that we are using now has context. It has a bigger picture and has a baseline level of image classification.

My approach uses a Vision Transformer model (ViT-B/16), which has been pretrained on SWAG weights. This combined with aggressive data augmentation, regularization, layered learning rates, AND test-time augmentation, resulted in the best achieved validation accuracy (using an 80/20 split) was 77%, and a Kaggle score of 85%.

2 Dataset

- **Classes:** 100 (folders, labeled 0–99)
- **Training images:** 1,000 total (about 10 per class)
- **Validation images:** 200 (80/20 split)
- **Training images used after split:** 800
- **Test images:** 1,000 unlabeled images

Label Mapping

The training data is organized into directories named 0 through 99. To ensure correct evaluation, the folder name is used directly as the integer class label... This is done in the dataset construction:

```
ds.class_to_idx = {cls: int(cls) for cls in ds.classes}
ds.samples = [
    (path, int(os.path.basename(os.path.dirname(path))))
    for path, _ in ds.samples
]
```

Preprocessing and Augmentation

All images are resized to 384×384 pixels to match resolution used during SWAG fine-tuning of the pretrained weights. ImageNet mean/standard deviation is used for normalization

Training augmentations (applied only to the training split):

- Random horizontal flip
- Random vertical flip ($p = 0.1$)
- Random rotation (± 20 degrees)
- Random perspective (distortion scale 0.2, $p = 0.3$)
- Random affine (translate ± 120 degrees, scale 0.9–1.1)
- Color jitter (brightness, contrast, saturation 0.4; hue 0.1)
- Random grayscale ($p = 0.1$)
- Gaussian blur (kernel size 3)
- Random erasing ($p = 0.3$)
- Mixup ($\alpha = 0.2$, applied per batch during training)

Validation and test images use only resize and normalization (so no augmentation). This ensures an unbiased evaluation. Separating Dataset objects with different transform pipelines enforces this. Both splits share file indices but different transform arguments.

3 Implementation

3.1 Model

Backbone: Vision Transformer ViT-B/16 (`torchvision.models.vit_b_16`) init with IMAGENET1K_SWAG_E2E_V1 pretrained weights. These weights are produced by 1) weakly supervising ViT-B/16 on 3.6 billion Instagram images (SWAG), and 2) e2e fine-tuning on ImageNet-1K at the 384 x 384 resolution, which achieves 85.3% top-1 accuracy. These, as a result, are the highest-quality pretrained features available in TorchVision. This makes them perfect for small dataset transfer learning since the features are generalized broadly.

Architecture changes: Only the classification head is replaced. The original 1,000 class layer is swapped for a 100 class layer:

```
model.heads.head = nn.Linear(model.heads.head.in_features, 100)
```

There is no additional pooling, dropout, or intermediate layers added since the ViT encoder already includes layer normalization BEFORE the head.

Fine-tuning: All parameters are trained end to end from the first epoch. Rather than freezing and unfreezing, layer-wise learning rates are used... The backbone (encoder and patch projection) has a 10x lower learning rate than the classification head. This prevents forgetting of pretrained features, but still allows the backbone to adapt to the changes.

3.2 Training

- **Loss:** Cross-entropy with label smoothing ($\epsilon = 0.1$)
- **Optimizer:** AdamW w/ layerwise LR:
 - Encoder + patch projection: $lr = 1 \times 10^{-5}$
 - Classification head: $lr = 1 \times 10^{-4}$
 - Weight decay: 1×10^{-4}
- **Scheduler:** Cosine annealing (`CosineAnnealingLR`, $T_{\max} = 60$)
- **Batch size:** 32
- **Epochs:** 60 (validation phase); 30 additional epochs on the full 1,000-image training set (full retrain phase). I feel there could have been fewer epochs for validation. Didn't have time to test though.
- **Mixed precision:** `torch.cuda.amp` (AMP) with `GradScaler` for faster training
- **Random seed:** 41 (fixed for random, numpy, torch, and torch.cuda)
- **Hardware:** NVIDIA T4 / A100 (Google Colab)
- **Framework:** PyTorch, TorchVision

The model is checkpointed when the validation accuracy improves. After the 60-epoch validation phase, the best checkpoint is loaded, and retrained on all 1,000 samples for 30 epochs to maximize test performance before submission. Again, early stopping would probably have been helpful here.

4 Experiments

Baseline

The initial baseline used EfficientNet-B2 with IMAGENET1K_V1 weights, trained with a two-phase strategy (5 frozen epochs + 50 fine tuning epochs) and 40 TTA runs. This achieved a Kaggle score of 0.77.

Hyperparameter Tuning

Validation accuracy on the 200 image held out split was used as the tuning signal throughout. The following were explored:

Change	Motivation	Effect
EfficientNet-B2 $\rightarrow$ ViT-B/16 SWAG	Stronger pretrained features	better val acc
Fixed val transform bug (no aug on val)	Unbiased evaluation	cleaner signal
Weight decay $10^{-2} \rightarrow 10^{-4}$	Less aggressive regularization	smoother convergence
Layerwise LR (backbone $10\times$ lower)	Prevent forgetting	stable fine-tuning
Added Mixup ($\alpha = 0.2$)	Regularize 10-sample/class regime	better val acc
Image size 224 $\rightarrow$ 384	Match SWAG fine-tune resolution	better val acc
AMP mixed precision	Speed ($2\times$ per epoch)	no accuracy change
Full retrain on all 1,000 samples	More training data for test	expected better Kaggle
TTA runs 40 $\rightarrow$ 60	More stable inference	marginal positive

Table 1: Ablations and their effects.

5 Results

Model / Setting	Best Val Acc	Kaggle Score
EfficientNet-B2 (baseline)	65%	0.77
ViT-B/16 SWAG (ours)	77.67%	84.54%

Table 2: Summary of results.

6 Discussion

What worked best:

1) Switching from EfficientNet-B2 to ViT-B/16 with SWAG weights was the single largest improvement. The SWAG pretraining on 3.6 BILLION images produces features that transfer far more effectively to unseen classes.

2) Mixup was the best regularization addition; with 10 samples per class, the model is obviously very prone to overfitting. So, mixup creates convex combinations of training pairs, which really expanded training distribution.

3) Fixing the validation transform pipeline (removing augmentation from the val set) was a correctness fix that also provided a cleaner signal for model selection.

Failure cases and observations:

1) Validation accuracy plateaued at around 77.67% and showed high variance between epochs, which is probably due to the small val set (200 images; each correct/incorrect prediction moves accuracy by 0.5%). This makes the val signal noisy for real generalization.

Limitations and future improvements:

- **Ensembling:** Using multiple models and averaging
- **External data / self-supervised fine-tuning:** With label scarcity, pretraining on unlabeled test images may further adapt features to the target distribution.

7 Reproducibility

- **Random seed:** 41, fixed for random, numpy, torch, and torch.cuda at the start of the script. `torch.backends.cudnn.deterministic = True` is also set.
- **Environment:** Google Colab, Python 3.12, PyTorch (see `print(torch.__version__)` output), TorchVision.
- **Data:** `train.zip` and `test.zip` from `/content/drive/MyDrive/cse144/`.

To train:

- ```
a) Mount Google Drive, then run all cells in order in untitled3.py / the Colab notebook.
b) The best checkpoint is saved to:
 /content/drive/MyDrive/cse144/best_model.pth (validation phase)
 /content/drive/MyDrive/cse144/best_model_full.pth (full retrain phase)
```

### To generate submission.csv:

- ```
a) Run the inference cell (TTA loop) after loading best_model_full.pth.
b) Output is saved to:
    /content/drive/MyDrive/cse144/submission.csv
```

8 Team Contributions

Samuel Morrow: All model design, implementation, training, hyperparameter tuning, and report writing.

9 References

1. TorchVision documentation and pretrained model weights.
<https://pytorch.org/vision/stable/models.html>
2. Yuyin Zhou, UCSC, Lectures and Resources